# PayInsight

## Compte rendu du projet : XML, XSD, XPath et service web SOAP en Java

Auteure : Bouassida Chayma

Date : 7 novembre 2026

Outils : Java, Maven, JAXP, JAX-WS, JAXB, JUnit 4, SoapUI, IntelliJ IDEA

# 1. Introduction et objectifs

PayInsight est une application Java construite en deux parties qui s'enchaînent. La partie 1 modélise un système de paiement simplifié en XML, le valide avec un schéma XSD et l'interroge avec XPath. La partie 2 expose ces mêmes données à des clients externes grâce à un service web SOAP. Ce compte rendu explique la modélisation, l'architecture, l'implémentation et les tests.

Objectifs :

- modéliser un domaine réel en XML et le décrire avec un schéma XSD ;
- valider le document et l'interroger avec XPath (partie 1) ;
- créer et publier un service web SOAP avec JAX-WS (partie 2) ;
- tester l'ensemble avec JUnit et SoapUI.

# 2. Modélisation XML (partie 1)

Le domaine choisi est le paiement électronique. Le document payments.xml a pour racine l'élément paymentSystem et contient cinq sections.

| Section | Contenu de chaque élément | Exemple |
|---|---|---|
| customers | id, name, email, country | C001, Ahmed Ben Ali |
| merchants | id, name, category, country | M001, TechStore |
| terminals | id, merchantId, location, status | T001, Sfax, active |
| transactions | id, customerId, merchantId, terminalId, amount (currency), date, status | TX001, 250.00 TND, approved |
| errorCodes | code, description, severity | 05, Transaction declined by issuer |

## Choix de modélisation

- **Une section par entité** : chaque type de données est regroupé dans un conteneur, ce qui rend les requêtes XPath simples.
- **Références par identifiant** : une transaction référence le client, le commerçant et le terminal au lieu de recopier leurs données.
- **Attributs et éléments** : les identifiants et la devise sont des attributs, le reste est en éléments.
- **Devise** : tous les montants sont en dinars tunisiens (TND).

## Exemple de transaction :

```
<transaction id="TX001">
  <customerId>C001</customerId>
  <merchantId>M001</merchantId>
  <terminalId>T001</terminalId>
  <amount currency="TND">250.00</amount>
  <date>2026-10-08</date>
  <status>approved</status>
</transaction>
```

## 3. Schéma XSD (partie 1)

Le fichier payments.xsd décrit la structure que payments.xml doit respecter. Il définit un type complexe nommé par entité (CustomerType, MerchantType, TerminalType, TransactionType, ErrorType), puis l'élément racine paymentSystem qui enchaîne les cinq sections dans un ordre fixe (xs:sequence). Chaque section accepte plusieurs éléments (maxOccurs="unbounded").

| Contrainte | Mise en œuvre |
|---|---|
| Identifiants uniques | attribut id de type xs:ID, obligatoire (use="required") |
| Références entre entités | customerId, merchantId et terminalId de type xs:IDREF |
| Montant | xs:decimal avec attribut currency obligatoire |
| Date | xs:date (format AAAA-MM-JJ) |
| Statut de transaction | énumération : approved, declined, pending |
| Statut de terminal | énumération : active, inactive |
| Gravité d'une erreur | énumération : low, medium, high |

## 4. Projet Maven et dépendances

Le projet a été créé dans IntelliJ IDEA avec l'archétype maven-archetype-quickstart (groupId com.payinsight, artifactId payinsight-xml, version 1.0-SNAPSHOT). La compilation se fait en Java 8 (maven.compiler.release) avec l'encodage UTF-8.

| Dépendance | Version | Rôle |
|---|---|---|
| junit | 4.13.2 (test) | Tests unitaires |
| jaxws-rt | 2.3.7 | Moteur du service SOAP (partie 2) |
| jakarta.xml.ws-api | 2.3.3 | API JAX-WS (partie 2) |
| javax.jws-api | 1.1 | Annotations @WebService (partie 2) |

```
PayInsight/
  data/payments.xml, payments.xsd
  src/main/java/com/payinsight/
    Main.java
    data/     Transaction, PaymentDataService
    service/  IPaymentService, PaymentService
    server/   PayInsightServer
  src/test/java/com/payinsight/PayInsightTest.java
  pom.xml, README.md
```

## 5. Validation XML/XSD (partie 1)

La méthode validateXML de la classe Main utilise SchemaFactory, Schema et Validator : la fabrique est créée avec XMLConstants.W3C_XML_SCHEMA_NS_URI, le schéma est chargé depuis le fichier XSD, un validateur est créé, puis validate() est appliqué au fichier XML. La méthode retourne true si le document est valide. Le programme principal n'exécute les requêtes XPath que dans ce cas.

## 6. Requêtes XPath (partie 1)

La méthode executeXPathQueries charge le document avec DocumentBuilder, crée un moteur avec XPathFactory, puis executeQuery compile chaque expression (XPathExpression) et affiche les nœuds trouvés.

| Objectif | Expression XPath | Résultat |
|---|---|---|
| Noms des commerçants | `/paymentSystem/merchants/merchant/name` | TechStore, FashionShop |
| Terminaux actifs | `/paymentSystem/terminals/terminal[status='active']/@id` | T001 |
| Transactions approuvées | `/paymentSystem/transactions/transaction[status='approved']/@id` | TX001, TX003 |
| Montant supérieur à 100 TND | `/paymentSystem/transactions/transaction[amount > 100]/@id` | TX001, TX002 |
| Transactions refusées | `/paymentSystem/transactions/transaction[status='declined']/@id` | TX002 |
| Description de l’erreur 05 | `/paymentSystem/errorCodes/error[@code='05']/description` | Transaction declined by issuer |

## 7. De la partie 1 à la partie 2 : cycle de vie du service

La partie 2 n’est pas un service isolé : elle expose les données de la partie 1. Le schéma ci-dessous reprend le cycle de vie d’un service web vu en cours (trois acteurs : l’annuaire, le serveur producteur et le client consommateur), appliqué à PayInsight.

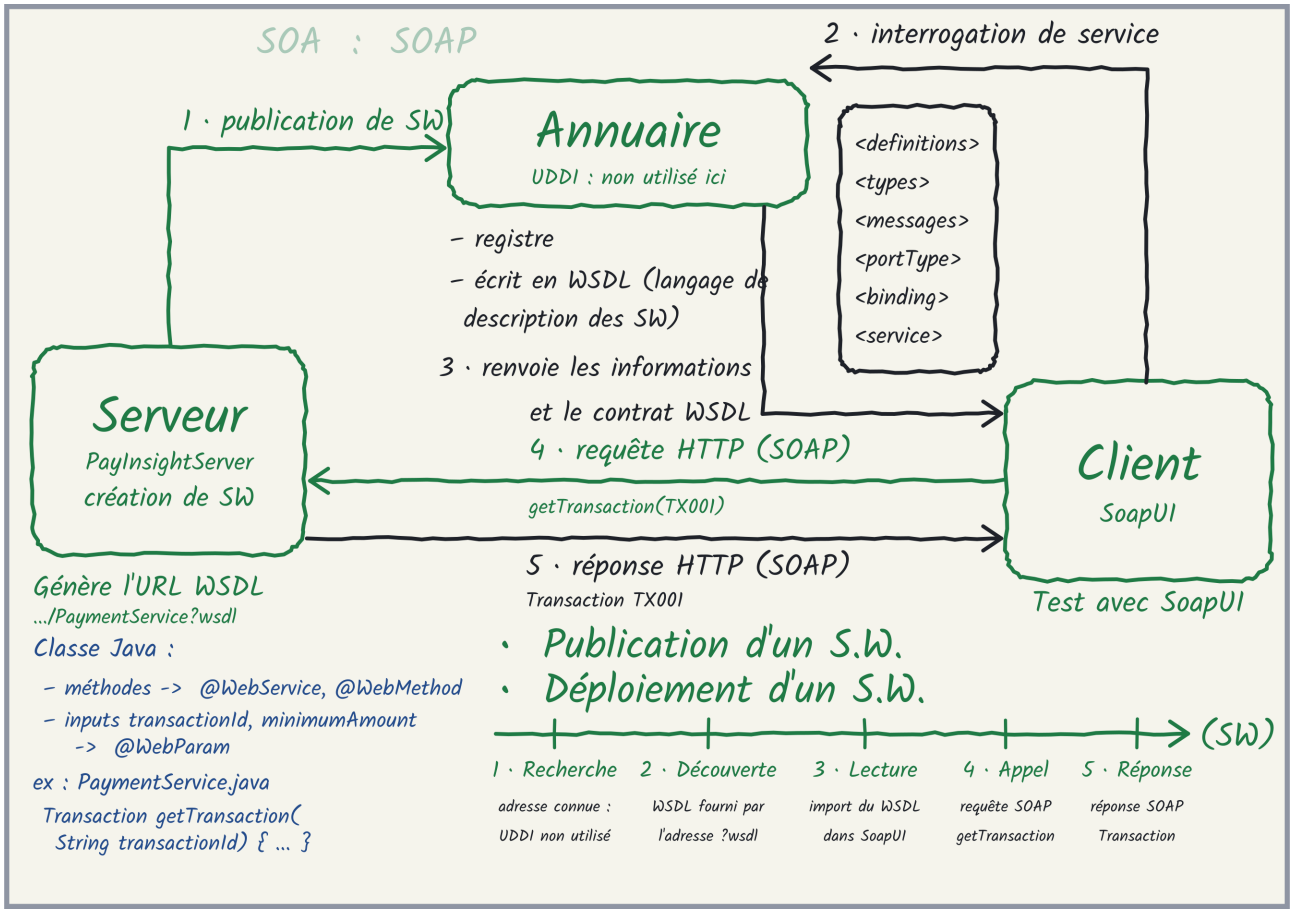



*Figure 1 : cycle de vie du service web PayInsight (SOA, SOAP)*

| Étape | Dans le cours | Dans PayInsight |
|---|---|---|
| 1. Publication | le serveur publie le contrat WSDL | PayInsightServer appelle Endpoint.publish ; le WSDL est généré à l'adresse ...?wsdl |
| 2. Interrogation / recherche | le client interroge l'annuaire UDDI | non utilisée : l'adresse du service est connue, il n'y a pas d'annuaire UDDI |
| 3. Découverte / lecture | l'annuaire renvoie le contrat WSDL | SoapUI importe le WSDL depuis l'adresse ...?wsdl |
| 4. Requête | le client envoie une requête HTTP (SOAP) | SoapUI envoie getTransaction avec transactionId = TX001 |
| 5. Réponse | le serveur répond par un message SOAP | réponse contenant la transaction TX001 (8 champs) |

## Annotations Java utilisées

- **Classe Java, méthodes** : @WebService déclare le service, @WebMethod expose chaque méthode comme opération.
- **Paramètres** : @WebParam(name = "transactionId") ou @WebParam(name = "minimumAmount") nomme le paramètre dans le message SOAP.
- **Exemple dans le projet** : Transaction getTransaction(String transactionId) dans IPaymentService, implémentée par PaymentService.

Chemin d'une requête SOAP :

- le client (SoapUI) envoie la requête au serveur PayInsightServer ;
- PaymentService, qui implémente IPaymentService, reçoit l'appel et contrôle les paramètres ;
- PaymentDataService appelle d'abord Main.validateXML() (partie 1), puis lit payments.xml ;
- il construit des objets Transaction, renvoyés au client dans la réponse SOAP.

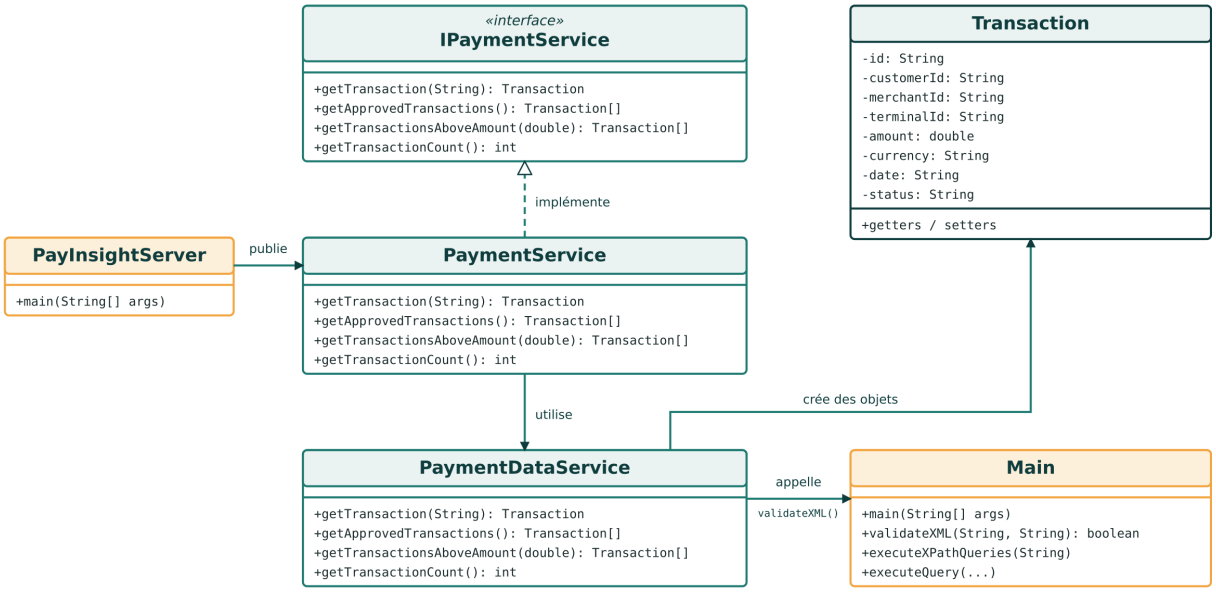



*Figure 2 : diagramme de classes du serveur*

# 8. Service web SOAP (partie 2)

## Composants

| Classe | Responsabilité |
|---|---|
| Main.java | Validation XML/XSD et requêtes XPath (partie 1) |
| Transaction.java | Objet transaction, annoté JAXB (@XmlAccessorType, @XmlType avec propOrder) |
| PaymentDataService.java | Lit payments.xml après validation et construit les transactions |
| IPaymentService.java | Contrat SOAP (@WebService, @WebMethod, @WebParam) |
| PaymentService.java | Implémente le contrat et contrôle les paramètres |
| PayInsightServer.java | Publie le service avec Endpoint.publish() |

## Structure du WSDL

Le WSDL généré par JAX-WS contient les six éléments vus en cours : une partie abstraite (types, message, portType) et une partie concrète (binding, service).

| Élément | Rôle | Dans PayInsight |
|---|---|---|
| types | types de données (XSD) | schéma importé depuis ...?xsd=1 |
| message | messages d'entrée et de sortie | 8 messages : 4 opérations, chacune avec une requête et une réponse |
| portType | opérations du service | IPaymentService : 4 opérations (input et output) |

| Élément | Rôle | Dans PayInsight |
|---|---|---|
| binding | protocole et format | PaymentServicePortBinding : SOAP, style document, use literal |
| service | adresse du service | PaymentServiceService, port PaymentServicePort, http://localhost:8086/PayInsight/PaymentService |

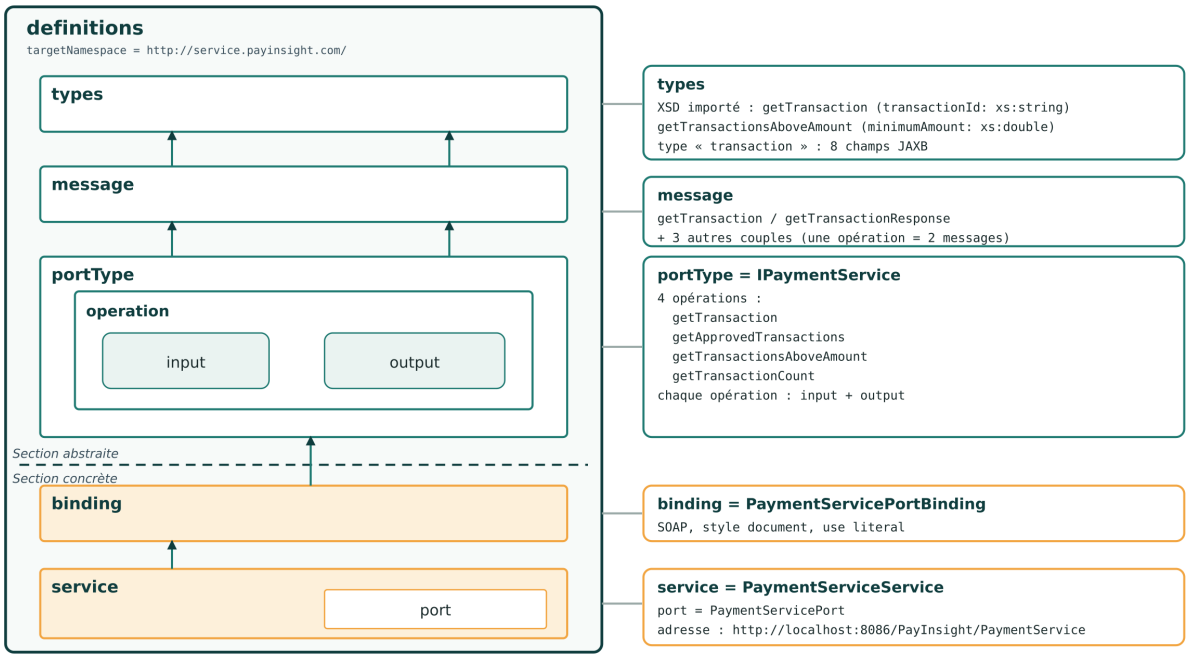



*Figure 3 : structure du WSDL de PayInsight*

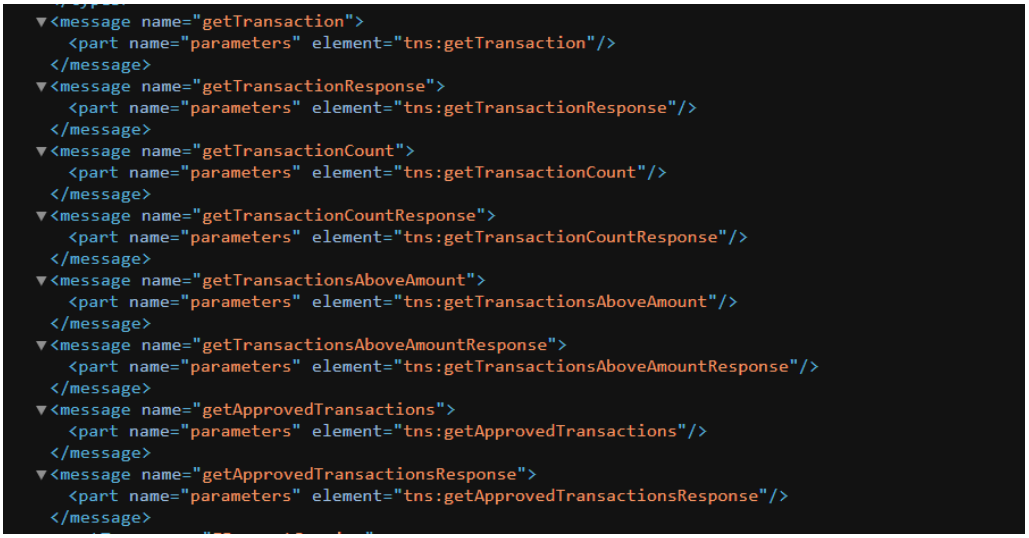



*Figure 4 : extrait du WSDL réel : types et messages (capture du navigateur)*

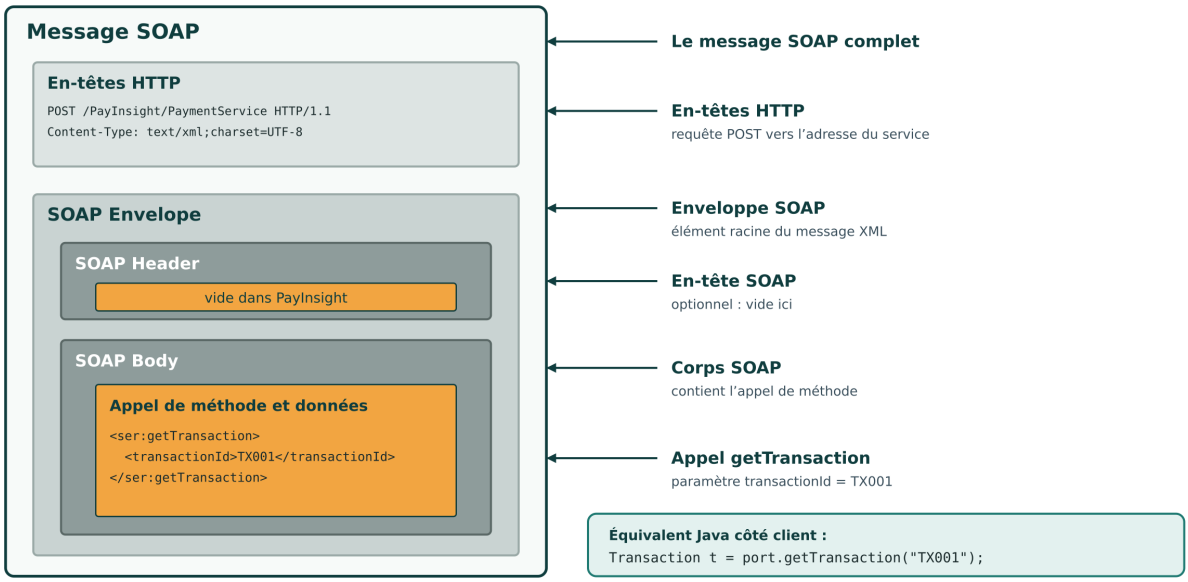



*Figure 5 : structure d'un message SOAP (getTransaction)*

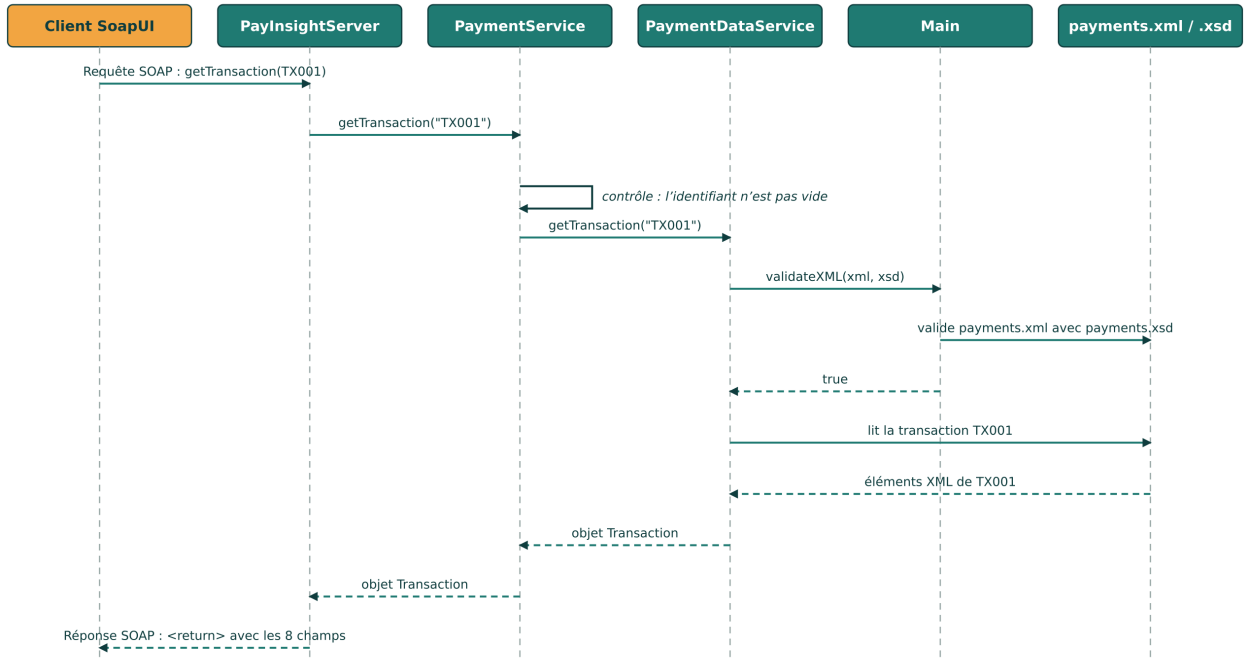



*Figure 6 : diagramme de séquence de getTransaction(TX001)*

## Opérations

| Opération | Paramètre | Résultat |
|---|---|---|
| getTransaction | transactionId | une transaction |
| getApprovedTransactions | aucun | les transactions approved |
| getTransactionsAboveAmount | minimumAmount | les transactions dont le montant dépasse ce minimum |
| getTransactionCount | aucun | le nombre total de transactions |

## Accès aux données et contrôles

- Chaque opération de PaymentDataService valide d'abord le XML avec Main.validateXML().

- Le DocumentBuilderFactory désactive le DOCTYPE et les entités externes, ce qui protège contre les attaques par entités externes (XXE).
- PaymentService lève une IllegalArgumentException pour un identifiant vide ou inconnu et pour un montant minimum négatif.

### Publication

```
String url = "http://localhost:8086/PayInsight/PaymentService";
Endpoint.publish(url, new PaymentService());
```

Le WSDL est disponible à la même adresse avec le suffixe ?wsdl ; SoapUI l'importe pour générer les requêtes. Le serveur doit rester lancé pendant les tests.

## 9. Tests et vérification

### 9.1 Exécution de la partie 1

Avec un document valide, la console affiche le succès de la validation puis les résultats des six requêtes, et le programme se termine avec le code 0.

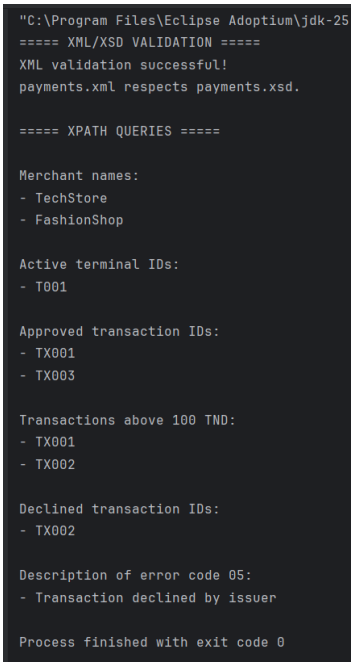



*Figure 7 : sortie console de Main*

Pour tester la validation, le statut approved de la transaction TX001 a été remplacé par unknown. La validation échoue (cvc-enumeration-valid), car la valeur n'appartient pas à l'énumération du schéma. La valeur valide a ensuite été restaurée.

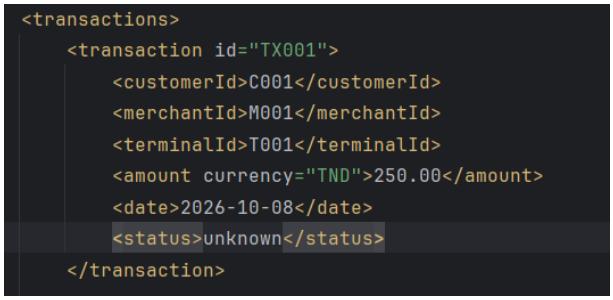



*Figure 8 : statut invalide dans le XML*

```
XML validation failed!
Schema validation error: cvc-enumeration-valid : La valeur 'unknown' n'est pas un facet valide par rapport à l'énumération '[approved, declined, pending]'.
```

*Figure 9 : message d'erreur de validation*

## 9.2 Tests SoapUI de la partie 2

Le service a été testé avec SoapUI en important le WSDL. Les quatre opérations ont été exécutées.

| Opération | Entrée | Résultat observé |
|---|---|---|
| getTransaction | transactionId = TX001 | TX001 : C001, M001, T001, 250.0 TND, approved |
| getApprovedTransactions | aucune | TX001 (250.0 TND) et TX003 (75.5 TND) |
| getTransactionsAboveAmount | minimumAmount = 100 | TX001 (250.0 TND) et TX002 (120.0 TND) |
| getTransactionCount | aucune | 3 |

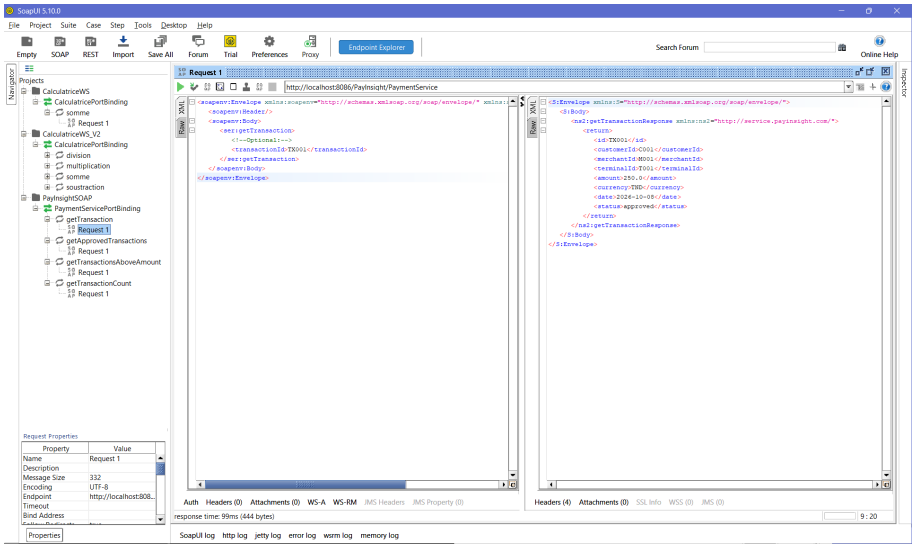


*Figure 10 : SoapUI, getTransaction (TX001)*

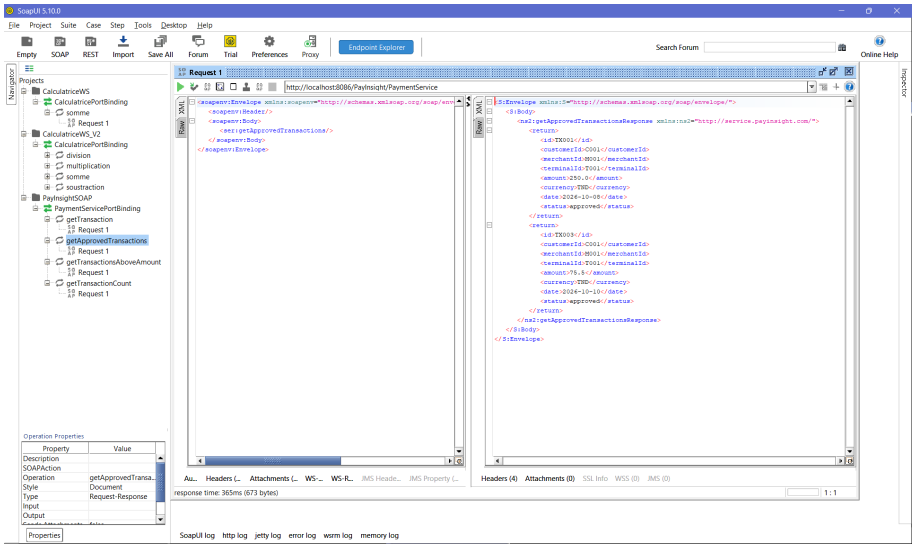


*Figure 11 : SoapUI, getApprovedTransactions*

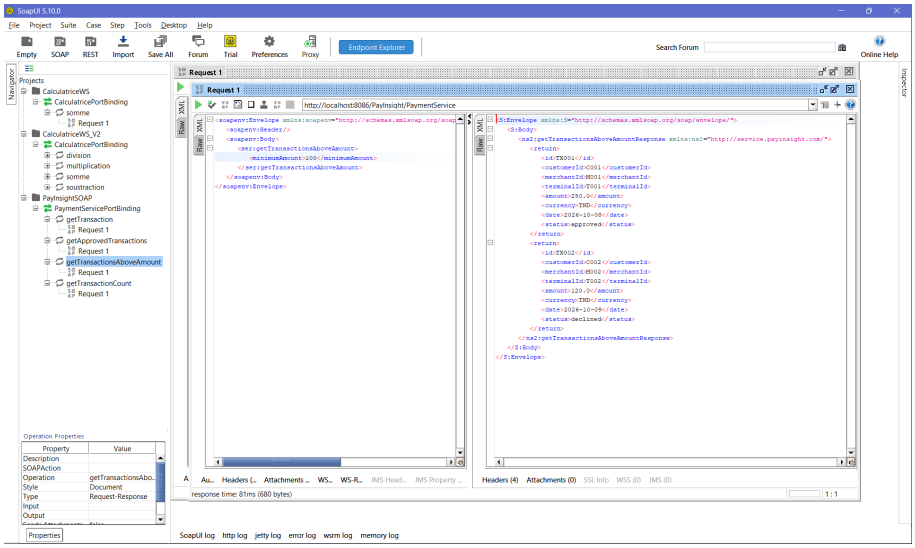


*Figure 12 : SoapUI, getTransactionsAboveAmount (minimum 100)*

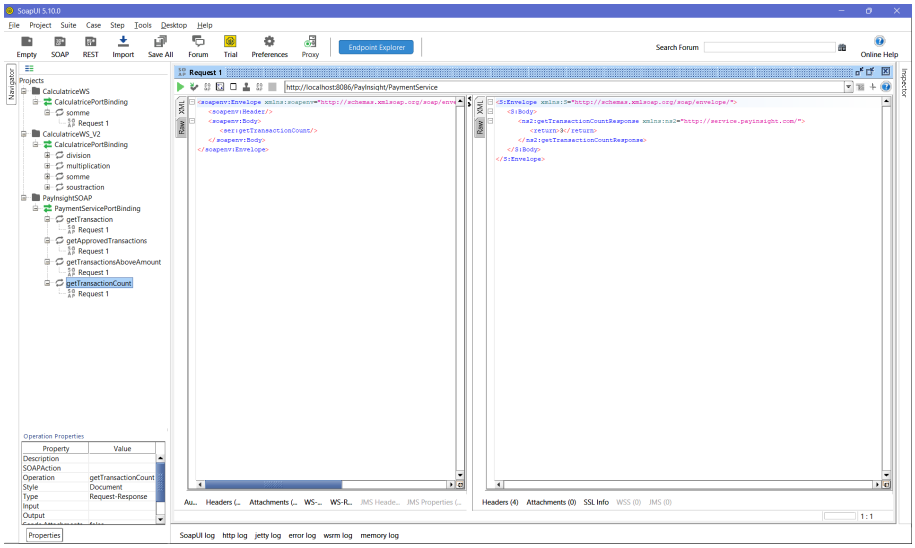


*Figure 13 : SoapUI, getTransactionCount*

## 9.3 Tests unitaires JUnit

Sept tests JUnit 4 sont regroupés dans PayInsightTest.

| Test | Vérification |
|---|---|
| testXMLValidation | le XML respecte le XSD |
| testGetTransaction | lecture de TX001 (client, montant, devise, statut) |
| testGetApprovedTransactions | 2 transactions, toutes approved |
| testTransactionsAboveAmount | 2 transactions de montant supérieur à 100 |
| testTransactionCount | 3 transactions au total |
| testInvalidTransactionID | TX999 lève une IllegalArgumentException |
| testNegativeAmount | un minimum négatif lève une IllegalArgumentException |

Résultat : Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, BUILD SUCCESS.

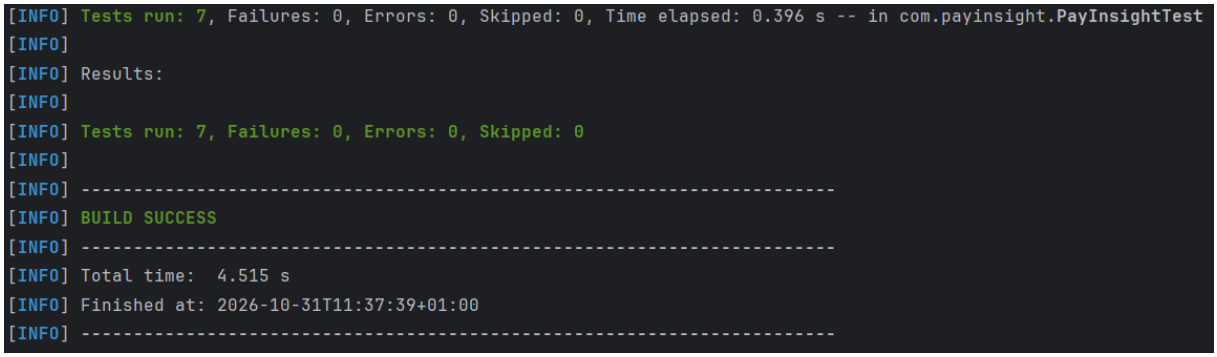


*Figure 14 : résultats JUnit*

## 10. Limites et pistes d’amélioration

- **Références non typées** : xs:IDREF accepte n’importe quel identifiant du document. Un customerId pourrait pointer vers un commerçant sans erreur. Les contraintes xs:key et xs:keyref permettraient de lier chaque référence à la bonne section.
- **Performance** : chaque opération valide et analyse payments.xml. De plus, getApprovedTransactions et getTransactionsAboveAmount appellent getTransaction pour chaque résultat, ce qui relance validation et analyse à chaque fois. Charger le document une seule fois par appel simplifierait le code.
- **Sécurité de Main** : le DocumentBuilderFactory de Main.executeXPathQueries n’est pas durci comme celui de PaymentDataService. Appliquer la même configuration partout serait plus cohérent.
- **Montants** : le montant est un double. Pour des montants financiers, BigDecimal évite les erreurs d’arrondi.
- **Annuaire et client** : l’annuaire UDDI n’est pas utilisé, et le seul client est SoapUI. Un client Java généré avec wsimport compléterait le cycle de vie.

## 11. Conclusion

PayInsight montre comment XML, XSD, XPath et SOAP s’articulent dans une même application Java. Le XML structure les données de paiement, le XSD garantit leur validité, XPath permet de les interroger, et le service SOAP les rend accessibles à des clients externes. JUnit et SoapUI confirment que l’ensemble fonctionne : sept tests réussis et quatre opérations SOAP vérifiées.